

SmartBancs App

Documento Técnico de Arquitectura

Reto Técnico NextGen Engineer — Tata Consulting Services

Autor: Erick Ballas | Fecha: septiembre 2026

1. Resumen Ejecutivo

SmartBancs App es una plataforma financiera de alta concurrencia diseñada para procesar hasta 10,000 transacciones por segundo (TPS) con una latencia máxima de 2 segundos por operación. El sistema integra un core bancario legado ("Bancs") a través de un mecanismo de sincronización asíncrono y un microservicio de Inteligencia Artificial que genera recomendaciones financieras personalizadas sin impactar el flujo transaccional crítico.

Este documento describe las decisiones arquitectónicas, el stack tecnológico seleccionado, la estrategia de integración con Bancs, el ciclo de vida del modelo de IA y el análisis del incidente simulado de concurrencia en pico de quincena.

2. Arquitectura del Sistema

2.1 Principios Arquitectónicos

- CQRS (Command Query Responsibility Segregation): Separación explícita de operaciones de escritura (transacciones) y lectura (saldos, recomendaciones) usando @nestjs/cqrs.
- Event-Driven Architecture: Las transacciones confirmadas publican eventos en RabbitMQ. Los consumidores secundarios (IA, sync Bancs) reaccionan de forma completamente asíncrona.
- Async-First para IA: El AI Worker nunca bloquea el flujo transaccional. Procesa recomendaciones en background y actualiza la transacción una vez disponible.
- Anti-Corruption Layer para Bancs: El sync-worker actúa como adaptador que traduce el modelo de dominio nuevo al formato del core legado, protegiendo ambos sistemas.
- Circuit Breaker: Si Bancs o la IA responden con latencia elevada, el sistema aísla el fallo y continúa operando en modo degradado.

2.2 Componentes del Sistema

Componente	Tecnología	Responsabilidad
Frontend Dashboard	HTML/CSS/JS + Nginx	UI para transferencias, stress test y visualización en tiempo real
Transaction API	NestJS (TypeScript)	REST API, validación DTO, CQRS CommandBus, métricas Prometheus
PostgreSQL 16	Base de datos relacional	Persistencia transaccional ACID, bloqueos pesimistas, índices por cuenta
Redis 7	Caché en memoria	Saldo en caché (write-through), lecturas < 1ms
RabbitMQ 3.12	Message Broker	Desacopla API de AI Worker y Sync Worker. Cola tx.completed
AI Worker	Python + Gemini API	Genera recomendaciones financieras async. Fallback mock si no hay API key
Sync Worker	Python	Sincroniza transacciones consolidadas al core Bancs en lotes
Prometheus	Monitoreo de métricas	Scraping de /metrics del API y AI Worker cada 15s
Grafana	Visualización	Dashboards de TPS, latencia p99, errores, cola RabbitMQ

2.3 Flujo de Datos Principal

Flujo Síncrono (< 2 segundos)

- 1. Cliente envía POST /transactions con accountFrom, accountTo, amount.
- 2. NestJS valida el DTO con class-validator (UUID, amount > 0).
- 3. CommandBus despacha ProcessTransactionCommand al handler.
- 4. Handler abre QueryRunner en PostgreSQL e inicia transacción ACID.
- 5. SELECT FOR UPDATE en orden canónico (UUIDs ordenados) — evita deadlocks cruzados.
- 6. Validación de saldo suficiente en la cuenta origen.
- 7. UPDATE balance origen (-amount) y destino (+amount).
- 8. INSERT en tabla transactions con status COMPLETED y traceId.
- 9. COMMIT — transacción completada en base de datos.
- 10. Write-through en Redis: actualiza saldo en caché con incrby/decrby.
- 11. EventBus publica TransactionCompletedEvent — NON-BLOCKING.
- 12. API retorna 200 OK con transactionId y traceId.

Flujo Asíncrono (no bloquea al cliente)

- 1. AI Worker consume mensaje "tx.completed" de RabbitMQ.
- 2. Llama a Gemini API con el monto de la transacción.
- 3. En caso de fallo de Gemini: activa fallback mock con 4 mensajes predefinidos.
- 4. UPDATE en tabla transactions con la columna recommendation.
- 5. Frontend polling /transactions muestra la recomendación al usuario.

3. Decisiones Técnicas Justificadas

3.1 NestJS como Framework de API

Decisión: NestJS + TypeScript sobre FastAPI (Python) o Spring Boot (Java).

Justificación: @nestjs/cqrs implementa el patrón CQRS de forma nativa, haciendo que la arquitectura sea legible directamente desde el código fuente. La estructura modular (TransactionsModule, AccountsModule, MessagingModule, ObservabilityModule) documenta sola el diseño sin necesidad de leer la documentación. El event loop de Node.js + adaptador Fastify provee ~30K req/s con I/O concurrente, suficiente para el escenario de 10K TPS.

Trade-off: Mayor boilerplate inicial vs FastAPI. Compensado con el CLI de NestJS para scaffolding automático.

3.2 PostgreSQL con Bloqueos Pesimistas

Decisión: SELECT FOR UPDATE con ordenamiento canónico de UUIDs.

Problema resuelto: En un sistema de 10K TPS con transferencias concurrentes, dos hilos pueden intentar transferir entre las mismas cuentas simultáneamente. Sin orden canónico, Hilo A bloquea cuenta X y espera cuenta Y, mientras Hilo B bloquea Y y espera X → deadlock.

Solución: Siempre bloquear los UUIDs en orden ascendente ([accountFrom, accountTo].sort()). Ambos hilos intentarán bloquear el mismo UUID primero → uno esperará, el otro completará. Elimina deadlocks cruzados sin sacrificar consistencia.

Complemento: PgBouncer en producción para connection pooling. maxQueryExecutionTime: 500ms configurado en TypeORM para alertar sobre queries lentos.

3.3 Redis como Caché Write-Through

Decisión: Saldo de cuenta en Redis, actualizado sincrónicamente con cada transacción.

Justificación: En picos de 10K TPS, leer el saldo desde PostgreSQL en cada transacción es inviable. Redis responde en < 1ms. El patrón write-through garantiza que el caché nunca quede desactualizado: se actualiza en el mismo request que modifica la base de datos. Política allkeys-lru con 256MB garantiza que las cuentas más activas siempre estén en memoria.

3.4 RabbitMQ como Bus de Mensajería

Decisión: RabbitMQ para el MVP. Kafka documentado como ruta de escalamiento.

Justificación: RabbitMQ permite levantar el ecosistema completo con Docker Compose en < 2 minutos, tiene UI de administración incluida (puerto 15672) y es suficiente para demostrar el patrón de desacoplamiento async. La migración a Kafka en producción agrega log de eventos persistente, replay de mensajes y throughput masivo — sin cambiar la interfaz de publicación/consumo.

3.5 Stack Políglota: TypeScript + Python

Decisión: API en NestJS/TypeScript, AI Worker y ETL en Python.

Justificación: Cada lenguaje se usa donde es más fuerte. Python domina el ecosistema de ML/IA (google-generativeai, pandas, asyncpg). TypeScript/NestJS domina el ecosistema web con decoradores y módulos. La comunicación entre servicios ocurre vía RabbitMQ — el lenguaje de implementación es un detalle, no un acoplamiento arquitectónico.

4. Integración con Core Bancs

4.1 Estrategia de Integración

El core legado "Bancs" no puede recibir el volumen transaccional directo de 10K TPS. La estrategia usa un Anti-Corruption Layer (ACL) con dos mecanismos complementarios:

- Batch Sync (ETL): El script `bancs_transform.py` procesa transacciones acumuladas en lotes periódicos (cada 5-15 minutos). Lee de la tabla `bancs_raw_transactions` en PostgreSQL, limpia y estandariza los datos, e inyecta hacia Bancs en una ventana de baja actividad.
- Event-Driven Sync: El `sync-worker` consume eventos de RabbitMQ y notifica a Bancs de operaciones críticas (montos elevados, tipos específicos) en tiempo cuasi-real, con reintentos automáticos y circuit breaker.

4.2 Pipeline ETL — `bancs_transform.py`

El script implementa las fases Extract-Transform-Load:

- Extract: Lee `bancs_raw_data.json` (simulación del dump de Bancs). Soporta JSON y CSV.
- Transform: `drop_duplicates` por `transaction_ref`, `dropna` en `account_from` y `amount`, normalización de fechas a ISO-8601 UTC con `pd.to_datetime`, limpieza de montos (strip de caracteres no numéricos), estandarización de `tx_type` a TRANSFER/DEPOSIT/WITHDRAWAL.
- Load: Inserta en tabla `bancs_raw_transactions` vía SQLAlchemy. Modo append para no perder historial. Log de registros procesados vs descartados.

4.3 Protección del Core Legado

- Rate Limiting: El `sync-worker` limita las llamadas a Bancs a un máximo configurable por minuto.
- Circuit Breaker: Si Bancs falla 3 veces consecutivas, el circuit breaker se abre y acumula operaciones localmente para procesar cuando Bancs se recupere.
- Idempotencia: Cada operación hacia Bancs incluye el `trace_id` de la transacción original para garantizar que reintentos no dupliquen registros.

5. Modelo de Inteligencia Artificial

5.1 Decisión del Modelo

Para el MVP se consume la API de Gemini (google-generativeai). El modelo gemini-flash-latest fue seleccionado por su baja latencia en texto corto y disponibilidad gratuita para desarrollo.

La restricción crítica del reto es que la IA no debe bloquear el flujo transaccional. La solución implementa el AI Worker como microservicio completamente separado que consume eventos de RabbitMQ después de que la transacción ya fue confirmada al cliente.

5.2 Ciclo de Vida del Modelo

- Alimentación: Transacciones procesadas quedan disponibles en PostgreSQL para análisis y futuro entrenamiento.
- Prompt Engineering: El prompt incluye el monto de la transacción y contexto de SmartBancs. Respuesta máxima de 2 oraciones para bajo consumo de tokens.
- Resiliencia: Timeout de 15 segundos en la llamada a Gemini. Si falla, activa fallback con 4 recomendaciones mock predefinidas. El AI Worker nunca propaga excepciones al flujo transaccional.
- Monitoreo: Contador Prometheus smartbancs_ai_requests_total con labels success/error. Detecta degradación del modelo en tiempo real.
- Escalamiento: El AI Worker puede escalarse horizontalmente añadiendo más consumers de la misma cola RabbitMQ. Distribución de carga automática con prefetch_count.
- Evolución: En producción, el modelo fundacional puede reemplazarse por un modelo fine-tuned sobre datos históricos de SmartBancs sin cambiar la interfaz del AI Worker.

5.3 Data Drift y Re-entrenamiento

Para un modelo productivo se implementaría monitoreo de data drift con Population Stability Index (PSI). Si $PSI > 0.25$ en distribución de montos o categorías de gasto, se activa pipeline de re-entrenamiento automático con MLflow para registro de versiones y rollback.

6. Observabilidad

6.1 Stack de Observabilidad

El sistema implementa las tres dimensiones de observabilidad (Pilares de la Observabilidad):

- Logs: NestJS Logger estructurado con contexto (TraceID, TransactionID, amounts). Python AI Worker usa logging.INFO/ERROR con formato estándar.
- Métricas: Prometheus scrapea /metrics del API (puerto 3000) y del AI Worker (puerto 8001). Métricas clave: smartbancs_transactions_total (counter por status), smartbancs_transaction_duration_seconds (histogram con buckets de 0.1s a 5s).
- Trazas: TraceID (UUID v4) generado en cada request y propagado a través de: API → PostgreSQL (columna trace_id) → RabbitMQ (campo en el mensaje) → AI Worker (log y DB).

6.2 Alertas Propuestas

Síntoma	Métrica	Umbral	Acción
Latencia alta	tx_duration_seconds p99	> 1.5s warning, > 2s critical	Escalar workers, revisar queries lentos
Error rate alto	tx_errors / tx_total	> 1%	Alerta PagerDuty, revisar logs de excepción
Cola saturada	queue_pending_messages	> 10,000	Escalar AI Worker consumers
Bancs degradado	bancs_sync_failures_total	> 3 en 5 min	Activar circuit breaker, acumular lote offline
AI degradada	ai_requests_total{status="error"}	> 10%	Activar fallback mock, alerta de degradación

7. Respuesta al Incidente Simulado

7.1 Descripción del Escenario

Escenario: Pico de quincena. 10,000 TPS simultáneos provocan: latencia severa (>5s), timeouts en PostgreSQL, deadlocks en tabla transactions, cola de RabbitMQ saturada, clientes reportando transferencias no completadas.

7.2 Diagnóstico (0-5 minutos)

Step 1 — Identificar procesos bloqueados en PostgreSQL:

```
SELECT pid, state, wait_event_type, wait_event, now()-query_start AS duration, query
FROM pg_stat_activity WHERE state != 'idle' AND wait_event_type = 'Lock' ORDER BY
duration DESC;
```

Step 2 — Detectar cadena de deadlocks:

```
SELECT blocked.pid, blocking.pid AS blocking_pid, blocked.query FROM pg_stat_activity
blocked JOIN pg_stat_activity blocking ON blocking.pid =
ANY(pg_blocking_pids(blocked.pid));
```

Step 3 — Queries más lentos (pg_stat_statements):

```
SELECT query, mean_exec_time, calls FROM pg_stat_statements ORDER BY mean_exec_time
DESC LIMIT 10;
```

7.3 Acciones de Mitigación Inmediata

- Matar proceso bloqueante: `SELECT pg_terminate_backend(<blocking_pid>);`
- Activar circuit breaker: Redirigir lecturas de saldo a Redis, no a PostgreSQL saturada.
- Reducir conexiones: Bajar `max_client_conn` en PgBouncer temporalmente.

- Pausar sync-worker: Eliminar competencia con el flujo transaccional.
- Habilitar modo degradado: Encolar transacciones, procesar cuando la presión baje.
- Escalar read replicas: Redirigir queries de lectura a réplica si está disponible.

7.4 Causa Raíz Identificada

El pico de quincena multiplicó por 8x el volumen normal. La ausencia de índice en `account_from+status` generó full table scans en validaciones de historial. Los full scans mantienen locks prolongados que degeneran en deadlock chains. El pool de conexiones sin límite permitió que 500+ conexiones simultáneas amplificaran el problema.

7.5 Acciones Correctivas Permanentes

Área	Acción	Prioridad	Estado
Código	Ordenamiento canónico de UUIDs implementado	ALTA	Implementado
DB	Índices en (<code>account_from</code> , <code>created_at</code> DESC) y (<code>account_to</code> , <code>created_at</code> DESC)	ALTA	En <code>schema.sql</code>
Infra	PgBouncer con pool por servicio	ALTA	Pendiente
Infra	Read replica PostgreSQL para lecturas	MEDIA	Pendiente
Proceso	Load testing automático antes de quincena	MEDIA	Pendiente
Proceso	Runbook documentado para picos	MEDIA	Pendiente

8. Gaps Identificados y Mejoras Propuestas

Gap	Impacto	Mejora Propuesta
sync-worker no implementado (<code>sync_worker.py</code> : 404)	No hay sincronización real con Bancs	Implementar consumer RabbitMQ que llame API Bancs con retry
Tests: directorio vacío (404)	Sin evidencia de calidad	Jest unit tests para <code>ProcessTransactionHandler</code>
Solo 2 cuentas en <code>seed.sql</code>	Datos de prueba limitados	Seed con 10+ cuentas de distintos balances y monedas
docs/ vacío (<code>arquitectura.md</code> : 404)	Entregable obligatorio faltante	Este documento cubre este gap
Sin autenticación JWT	Riesgo de seguridad	NestJS Guard con JWT para endpoints transaccionales

9. Conclusiones

SmartBancs App implementa una arquitectura event-driven que cumple los requisitos críticos del reto: procesamiento transaccional < 2 segundos, IA no bloqueante, protección del core Bancs y observabilidad completa. Las decisiones tecnológicas están alineadas con contextos financieros de producción real: ACID en PostgreSQL, consistencia eventual para recomendaciones de IA y resiliencia ante fallos de servicios externos.

El uso del patrón CQRS con `@nestjs/cqrs` hace que la separación de responsabilidades sea explícita en el código fuente, facilitando la mantenibilidad y escalabilidad horizontal de cada componente de forma independiente.